

TCNA - Exercise 7

Due July 25, 2024

1 Numerical simulation of the associative memory model

In this exercise you will implement a numerical simulation of the model described in class. Given a network of N neurons, P memory patterns are used to define the following connectivity matrix:

$$J_{ij} = \frac{1}{Nf(1-f)} \sum_{\mu=1}^P (p_i^\mu - f)(p_j^\mu - f) ; J_{ii} = 0 \quad (1.1)$$

where we define the memory patterns by assigning each $p_i^\mu = \begin{cases} 1 \\ 0 \end{cases}$ with probability $\begin{matrix} f \\ 1-f \end{matrix}$.

The dynamics are:

$$\tau \dot{r}_i = -r_i + g(h_i) \quad (1.2)$$

Where:

$$h_i = \sum_j J_{ij} r_j$$
$$g(x) = \frac{1}{1 + e^{\beta(\theta - x)}}$$

Throughout this question, set $N = 1000$, $f = 0.5$ and $\theta = 0$.
(We recommend setting $\Delta t \leq \frac{\tau}{20}$ for simulations with Euler method)

1.1 Stability of Memory Patterns for Large- β

In order to mimic the large- β limit, set β to some large value like $\beta = 20$.

- For $P = 10, 20, 30, 80, 90, 100, 200, 300, 400, 500$, we would like to know how stable the memory patterns are. In order to do so, select a random sample of the memory patterns ($P_{\text{smp}} = 20$) and for each of these memory patterns μ start the network from the initial condition of that memory pattern, that is $\forall i \ r_i(0) = p_i^\mu$, run the dynamics for short time $T = 0.25\tau$ and check the fraction of neurons that have errors (defined by, for example, $|r_i(T) - p_i^\mu| > \epsilon_{\text{err}}$ for $\epsilon_{\text{err}} = 0.1$). Plot the probability of an error vs P .

We saw in class that for large N , and given the initial condition of memory pattern μ , the input to neuron i is normally distributed $h_i \sim \mathcal{N}(p_i^\mu - f - \theta, \frac{Pf}{N})$. The probability of an error to a single neuron can be calculated analytically using:

$$Pr(h_i < 0 | p_i^\mu = 1) = \int_{-\infty}^{-\mu/\sigma} \frac{dx}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$
$$Pr(h_i > 0 | p_i^\mu = 0) = \int_{-\mu/\sigma}^{\infty} \frac{dx}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

- Plot the analytical expression for the probability of a single error as a function of P , and show that for short timescales the analytical expression predicts the stability of the memory. To do so, you might want to use the error function (`erf()` in Matlab, or `scipy.special.erf()` in Python) within your calculation¹.
- Repeat the numerical simulations, this time for intermediate ($T = 2\tau$) and long ($T = 18\tau$) simulation time. On the same graph, plot the probability of a single error vs P for short, intermediate and long simulation time, as well as the analytical result. Try to suggest a qualitative explanation to the difference between the analytical calculation and the numerical results for long timescales. How does this difference depends on P ?

1.2 Overlap Dynamics and β - Dependence

We will now fix P and explore the stability of the memory patterns with different values of β . In order to do so, start the network at the initial condition of one memory pattern. We are interested in the evolution with time of the overlap of the state of the network with each memory pattern μ , defined by:

$$m^\mu = \frac{1}{Nf(1-f)} \sum_j (p_j^\mu - f)r_j \quad (1.3)$$

1. Set $P = 50$. Plot the value of the overlap with each of the 50 memory patterns on the same graph. Mark in particular the overlap with the memory pattern that was the initial condition (the first one) in some fashion (different color or thicker line). Perform this procedure for $\beta = 1, 4, 6, 8, 10, 15$. For all values of the β keep the *same* memory patterns, which also means the same connectivity matrix. Create a separate plot for each of the β values.
2. For values of $\beta = 1, 2, 4, 6, 8, 10, 15, 20, 25$, repeat the above simulation for 10 trials, each one with a different memory pattern as the initial condition. Plot the average final value of the overlap as a function of β .
3. Repeat the 10 trials for the same set of β values but this time with $P = 65$. Plot the average final overlap as a function of β .

1.3 Associative Memory Recall

We will now attempt to recall memory patterns by setting initial conditions which are only partially overlapping with one of the memories. In each trial we will start the initial condition with a fraction f_{err} of errors, i.e. for some μ set $r_i(t=0) = p_i^\mu$, and then take the first $f_{err}N$ neurons and set them to $r_j(0) = 1 - p_j^\mu$.

1. First set $\beta = 20$. For $P = 50$, choose 20 different memory patterns, and run trials for values of $f_{err} = 0.05, 0.1, \dots, 0.45$. Define a correct recall of a memory pattern to be up to 5% error. Plot both the average probability of single-neuron error and the probability of correct memory-recall vs f_{err} .
2. Repeat the above trials for $\beta = 10$ and $\beta = 15$. Again, plot average probability of single-neuron error and probability of correct recall vs f_{err} .

Qualitative Section

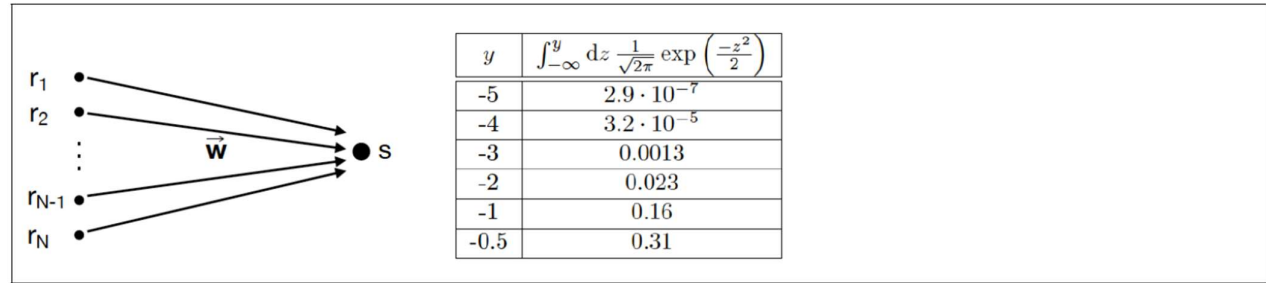
Discuss your observations from each of the simulation sections above, in particular relating to the dependence of the network on both β and P .

Describe what we mean by “associative memory” in the context of the model. How well would you say the networks performed associative memory in your simulations? Are there aspects of associative memory that are missing from this model?

¹Read about the error function to understand how it should be used in this specific calculation

2 Modified exam question (2018 Moed A)

Part 1.1 (15 Points)



We consider a two-layer feedforward network (see above) in which an input layer of N neurons, r_i , projects to a single output neuron, s . We consider P input patterns:

$$p_i^\mu = \begin{cases} 1 & \text{with probability } f \\ 0 & \text{with probability } 1 - f \end{cases}$$

and associate to each pattern a desired output $q^\mu = \begin{cases} 1 & \text{with probability } f \\ 0 & \text{with probability } 1 - f \end{cases}$

We connect input to output via a Hopfield-like vector of synaptic weights, $\vec{w}$:

$$w_j = \frac{1}{f(1-f)N} \sum_{\mu=1}^P (q^\mu - f)(p_j^\mu - f)$$

The activity of the output layer is:

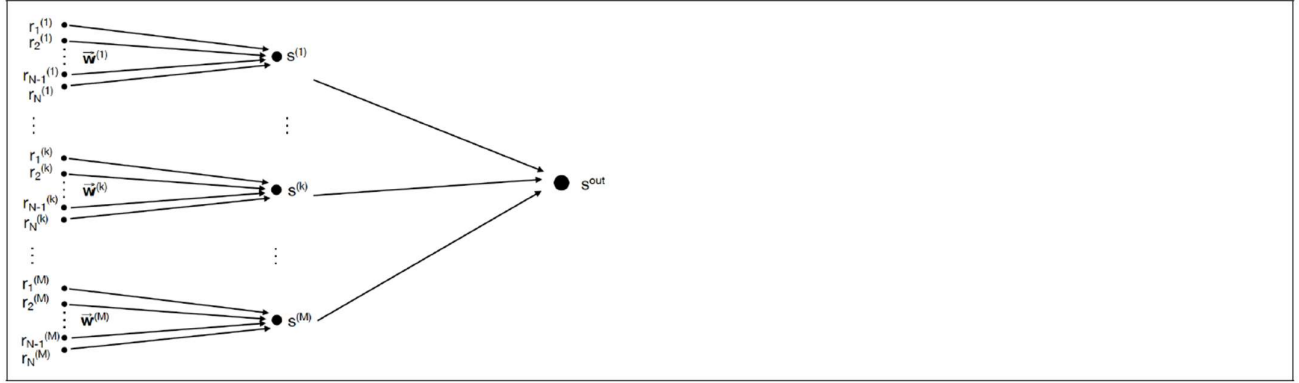
$$s = H(h - \theta)$$

Where H is the Heaviside function and $h = \sum_{j=1}^N w_j r_j$.

Suppose that $r_i = p_i^1$.

- Choose θ for which $P(s = 1 | q^1 = 0) = P(s = 0 | q^1 = 1)$.
- Write the expression for the probability that the output neuron has an error, i.e. $\text{Pr}^- \equiv P(s \neq q^1)$.
- Suppose that $P = 1000, N = 400, f = 0.1$. Calculate Pr^- . Use the table above.

Part 1.2 (15 Points)



Now we consider M copies of the feedforward network as pictured, with the first memory pattern common to all networks and the rest local to every network.

Each network, k , has an input layer of N neurons, $r_i^{(k)}$, projecting to a single output neuron, $s^{(k)}$:

$$s^{(k)} = H \left(\sum_{j=1}^N w_j^{(k)} r_j^{(k)} - \theta \right)$$

And θ is the same one that was found in the previous section.

The synaptic weights $\vec{w}^{(k)}$ for the network k is given by:

$$w_j^{(k)} = \frac{1}{f(1-f)N} \sum_{\mu=1}^P (q^{\mu(k)} - f)(p_j^{\mu(k)} - f)$$

Where $p_i^{\mu(k)}$ is the μ th memory pattern in the network k , and $q^{\mu(k)}$ is the corresponding desired output.

The shared memory pattern has $p_i^{1(k)} = p_i^1$ and $q^{1(k)} = q^1$ for all networks k , but $p_j^{\mu(k)}$ and $q^{\mu(k)}$ for all other memory patterns $\mu > 1$ are drawn independently for each network k .

Finally, we define a readout, s^{out} , whose input is the average of all M output neurons:

$$s^{out} = H \left(\frac{1}{M} \sum_{j=1}^N s^{(k)} - \theta^{out} \right)$$

We set all the input layers to the first memory pattern: $r_i^{(k)} = p_i^1$

(A) Assume that M is large and write the expression for $Pr_{out}^- \equiv P(s^{out} = q^{\mu})$ as a function of M . Choose θ^{out} for which $P(s^{out} = 1 | q^1 = 0) = P(s^{out} = 0 | q^1 = 1)$.

(B) Suppose $P = 1000, N = 400, f = 0.1, M = 25$ and calculate (approximately) Pr_{out}^- . Use above table.

Part 1.3: Qualitative Section (5 points)

What is the impact on the probability of error due to reading-out from multiple networks rather than just one? Why?